

Write-up Assignment-2

- ⇒ First i imported turtle library & random library.
- ⇒ Turtle library to use turtle graphics & random for shuffling numbers.
- ⇒ Then i made count=0 to count moves of user & a empty list named move-list that will we use to show replay later.

Start function :

- ⇒ This function i made for starting the game.
- ⇒ In this first ~~we~~ i called global variables count, board, moves-list, shuffling.
- ⇒ All other function is in this function, in fact whole code will be in this function.
- ⇒ Then we defined count=0 & moves-list=[]
- ⇒ Then ~~he~~ created ³ turtles two, counter & res and also made scr for screen.
- ⇒ Then i increased speed of counter & res & hid the counter & res.
- ⇒ Then by res i wrote 'Reset' in left corner that will become our reset button.

* Reset function :

- ⇒ In this we will get two inputs by clicking on screen that will be coordinates of the point that we clicked.
- ⇒ If we clicked on $x \in [-300, 300]$ & $y \in [200, 240]$ then screen will clear & start function will

run again.

- ⇒ Then i used `scr.onclick (reset)`. It will take input from screen in which it will be get click. & it will run reset function.

* draw_grid function :

- ⇒ This function is made to make grids & write numbers in grid.
- ⇒ First it will write 'Moves taken = ' & moves that we took in upper right corner.
- ⇒ Then i made `scr.tagger(0)` to make shifting of grid smoother.
- ⇒ Then by `twr.goto(-200 + (x * 100), 200 - (x * 100))` twr will go to upper left corner of particular grid. grid will decided by (x, y) that is row number & column number.
- ⇒ Then by for loop i made square box of grid.
- ⇒ Then i used `board[r][c]` first this i made by numbers from 1 to 15 & None but it is randomly shuffled according to difficulty.
- ⇒ If the element is not None that it will write number in centre of grid.
- And at last there is `scr.update(1)`, it will update the screen to latest structure.

* win function :

- ⇒ This function is made to check win condition
- ⇒ There is list named `coo` which has 1 to 15 numbers in correct order
- ⇒ Then we made a other list `flist` which has

current board's structure.

- If flag will be equal to 0 then screen will clear and it will show that "Solved!!" message at center of string.
- And also it will show moves taken at bottom right corner.
- After that, i made textinput which asks if you want to play again or not.
- If we type yes then screen will clear again and Start function will again run.
- else it will ask if you want to watch replay or not.
- If you type yes then it will call play-replay function which i made after.
- else it will close the game.

* Move-up function:

- This function is used to move numbers to its upper position if there is none.
- This will only work for $x < 3$. if $x < 3$ then it will swap `board[x][c]` & `board[x+1][c]`.
- And this will also increment count by one.
- And if we use this then "Up" will append in `moves-list` and also reduces goal by `down-goal`.
- And if it is not shuffling & not replaying, then it will check win function.

* move-down :

- Same as move-up but it will append "Down" instead of "Up". And only work if $x > 0$.

* Move-left:

→ Same as move-up but append "Left" & works when $c < 0$

* move-right:

→ Same as move-up but append "Right" & works when $c > 0$

* Play-replay function:

- This is a function to play replay.
- First it will make copy of shuffled board. and store it in board.
- Then it will again make it by draw-grid.

* Step function:

- If $\text{index} < \text{len}(\text{move-list})$ then move will become $\text{move-list}[\text{idx}]$ (we appended moves in moves-list)
- Then it will run according to it in some time & again run with $\text{idx}+1$.
- If $\text{idx} \geq \text{len}(\text{move-list})$ then it will write "replay finished"
- Then i. set background image & turtle color etc.
- Then it will fill light yellow color in background of board.
- Then there is two for loops which will create 4×4 grids

- Then i made numbers which has 1 to 15 numbers & one None.
- And id board [0][0]
- Then i made 2D list board which will store numbers in 2D way
- then if board [0][0] is not none then it will print it on centre of grid.
- Then it asks about difficulty whether you want to play easy, medium or hard
- Then i made dictionary moves which have easy: 30, medium: 60 & hard: 120.
- Then i created last-move - none which will store our last move.
- then made list all-move which have function of up, left, right, down
- and opp which have opposites of each move
- And then i started shuffling
- for i in range (moves) which has value from dictionary.
- choices will be element from all-moves.
- If last-move & opp [last-move] will be in choices, then it will remove opp [last-move] to prevent same move to ~~be~~ nullify.
- Then move will be equal to randomise of choices
- And last-move will be move.
- For controls `os.system('clear')` is used.
- & on key "Up" will call move-up, "Down" will call move-down, "Left" will call move-left & "Right" will call move-right.
- Then i called start function to start the game.